The autoencoder lab: the same loop, no labels

AI Methods for Science (77222) · Part I, week 4; continues `lab_cnn.pdf`

The setup

`ae_lab.py` (from https://github.com/yohai/aims-lab-session2; next to `mlp_lab2.py`, which it imports) has the same `train(config)` interface as `cnn_lab.py`; the result has `"config"`, `"history"`, `"summary"` and `"model"`. The target is the input itself. The network is a convolutional encoder (three stride-2 conv blocks, $32 \to 16 \to 8 \to 4$ pixels, then a linear layer to d numbers, the latent vector) and the mirror-image decoder (a linear layer, then transposed convs back to $3 \times 32 \times 32$). The loss is the mean squared error per pixel between the image and its reconstruction, pixels in $[0, 1]$. The summary reports `val_mse` (best epoch), `train_mse`, `n_params`, `sec_per_epoch`. The keys:

key	default	meaning
`latent`	`32`	the size d of the bottleneck: 3072 numbers go in, d come out of the encoder
`channels`	`[32, 64, 128]`	channels of the three encoder blocks; the decoder mirrors them
`noise`	`0`	standard deviation of Gaussian noise added to the *input* during training; the target stays clean (E25)
`exclude_class`	`-1`	a class 0..9 removed from the training and validation sets; -1 for none (E24)
`optimizer`, `lr`, `weight_decay`, `schedule`, ...	AdamW, 10^{-3}, 0.05, cosine	as in `mlp_lab2.py`
`epochs`	`10`	
`device`	`"auto"`	GPU (`cuda`) or Apple GPU (`mps`) if present, else CPU

The helpers, all on the validation split unless they take images:

call	returns
`load_pixels()`	`x_tr, y_tr, x_va, y_va`: images $(N, 3, 32, 32)$ in $[0, 1]$ and labels; 45,000 / 5,000
`reconstruct(r, x)`, `encode(r, x)`	$\hat{\mathbf{x}}$, same shape as `x`; the latent vectors, (N, d)
`score(r, x)`	the reconstruction error of each image, $\|\mathbf{x} - \hat{\mathbf{x}}\|^2/3072$, $(N,)$
`pca_mse(d)`, `pca_reconstruct(x, d)`, `pca_scores(x, d)`	the same for PCA with d components, fitted on the training images: validation MSE, reconstructions, the d coordinates. No training; the first call takes ≈ 5 s
`anomaly_scores(r)`	`score` of up to six sets, a dict: `"val"`, `"excluded"`, `"permuted"`, `"noise"`, `"constant"`, `"eurosat"` (E24)
`auroc(normal, anomalous)`, `roc_curve(normal, anomalous)`	the area under the ROC curve; the false-alarm and detection rates for a plot

Compute

Default network, 352 steps per epoch: ≈ 4 s per epoch on an Apple M-series GPU, ≈ 35 s on an M5 CPU, about twice that on an older laptop CPU; d barely changes the cost. **On a CPU, run 5 epochs instead of 10 and say so.** E23 and E25 reuse the networks of E22: keep the results in memory (a notebook), or save each with `torch.save(r, "ae_d32.pt")` and load it with `torch.load("ae_d32.pt", weights_only=False)`. Rules as before: do not edit the script; predict before each run, in writing.

No report

Nothing from this sheet is submitted. E22–E24 are done in class, E25 is optional. The Collect and Answer lines say what to look at and what to decide, as before; so does the rule: predict, in writing, before each run.

The experiments

E22. How small can the bottleneck be? ~3 min on a GPU

Run. `latent` $\in \{8, 32, 128, 512\}$, 10 epochs each; `pca_mse(d)` for the same d, and for as many other d as you like (it costs nothing).

Collect. validation MSE vs. d, the network and PCA on one log–log axis; a table with `n_params`, `val_mse`, `train_mse`, `sec_per_epoch`; one figure of 8 validation images: the originals, then a row of reconstructions for each d, network and PCA (`reconstruct`, `pca_reconstruct`).

Compute. four runs of 10 epochs: ≈ 3 min on an Apple GPU; on an M5 CPU, 5 epochs each, ≈ 12 min.

Answer. Before running: sketch the two curves. Afterwards: PCA is the best *linear* encoder–decoder pair there is. Does the network beat it, and at which d? What could a nonlinear network represent that PCA cannot, and does the MSE show it? What is the smallest d at which you can still name the class from the reconstruction? Why are the reconstructions blurred rather than wrong in some other way?

E23. Is the latent vector good for anything else? ~3 min

The encoder was trained to reconstruct, nothing else. Freeze it and ask a different question of its output: can a linear classifier, trained on the d latent numbers of each training image, name the class? To know whether the answer is any good, run the same classifier on two other descriptions of the image that involve no network at all: the 3072 raw pixels, and the d PCA coordinates.

There is no helper for this one; the classifier is yours to build. It is multinomial logistic regression: one `nn.Linear(k, 10)` from the k features to 10 logits, cross-entropy loss, trained on the features of the 45,000 training images, evaluated as accuracy on the 5,000 validation images. Two details that matter: standardize each feature with the mean and standard deviation of the training set before training, otherwise the scales of the latent numbers set the learning rate for you; and the whole feature matrix fits in memory, so there is no need for minibatches: a few hundred steps of full-batch Adam at `lr=1e-2` on the whole $45{,}000 \times k$ matrix are enough. Check that the classifier is converged, i.e. that more steps do not change the validation accuracy.

Run. the classifier on three kinds of features, `x_tr, y_tr, x_va, y_va = load_pixels()`:

- the network: `encode(r, x_tr)` and `encode(r, x_va)` for each of the four networks of E22 ($d = 8, 32, 128, 512$);
- PCA: `pca_scores(x_tr, d)` and `pca_scores(x_va, d)` for the same four d;
- the pixels: `x_tr.reshape(len(x_tr), -1)` and the same for `x_va`, once.

Collect. classifier accuracy vs. d: the network and PCA as two curves, the pixels as a horizontal line; two more horizontal lines for reference, the MLP on pixels of `lab_mlp.pdf` (53%) and the supervised CNN of E16 (71%).

Compute. seconds per classifier on a GPU; on a CPU ≈ 10 s for the pixels and ≈ 20 s per network, most of it encoding the 45,000 images.

Answer. Before running: the order of pixels, PCA and the network at each d. Afterwards: labels enter here for the first time, and only in the linear classifier. Can a classifier on PCA coordinates ever beat the classifier on the pixels? Why? Network minus pixels is what the structure of $\mathbf{x}$ alone buys; CNN minus network is what the labels buy. Does the ranking on classifier accuracy agree with the ranking on MSE in E22?

E24. Hold out a class, then score everything

~3 min on a GPU

The idea: train the autoencoder on nine of the ten classes, so that the tenth is something it has never seen. Then feed it images of several kinds and use the reconstruction error of each image, $s(\mathbf{x}) = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$, as an anomaly score: the hypothesis is that what the network has never seen it cannot reconstruct, so s should be high on the held-out class and on anything else that is not a natural image. `anomaly_scores(r)` computes s for six sets of images and returns a dict with one array of scores per set:

- `"val"`: validation images of the nine training classes. These are the normal data; every other set is compared against them.
- `"excluded"`: 1000 images of the held-out class, which the network never saw.
- `"permuted"`: validation images with their pixels shuffled, the same fixed permutation for every image (E16's trick). The same pixel values, no spatial structure.
- `"noise"`: images of uniform random noise.
- `"constant"`: images of a single colour, one random colour per image.
- `"eurosat"`: satellite patches from E21, resized to 32×32. Natural images, but not of the kind the network trained on. Present only if `data/eurosat_uint8.pt` exists, i.e. if you ran `python eurosat_lab.py -download`.

A detector is judged by `auroc(normal, anomalous)`: the probability that a random image of the anomalous set scores higher than a random normal image. 1 is a perfect separation, 0.5 is chance, and *below* 0.5 means the anomalous images reconstruct better than the normal ones.

Run. three networks, 10 epochs each:

- `train({"latent": 32, "epochs": 10, "exclude_class": 6})`: frog held out, a class that looks unlike the other nine;
- `train({"latent": 32, "epochs": 10, "exclude_class": 9})`: truck held out, a class that looks much like one that stays in (car);
- `train({"latent": 512, "epochs": 10, "exclude_class": 6})`: frog again, with a bottleneck 16× wider.

For each network, `s = anomaly_scores(r)`, then `auroc(s["val"], s[k])` for every key `k` other than `"val"`.

Collect. for the first network, a histogram of $\log s$ for each of the six sets, all on one axis; the ROC curve of the held-out frogs against `"val"` (`fpr, tpr = roc_curve(s["val"], s["excluded"])`); a table of AUROC, one row per set, one column per network.

Compute. the nine-class training set is 10% smaller, so slightly under the default cost: three runs of 10 epochs, ≈ 2 min on an Apple GPU; on an M5 CPU, 5 epochs each, ≈ 8 min. Scoring: seconds.

Answer. Before running: write down the order in which you expect the six sets to score, highest first (you did this in class; it is the hypothesis). Afterwards, for the first network: which sets does the score separate from the normal data (AUROC near 1), and which does it not? Which sets come out *below* 0.5, and what do these images have in common that makes them easy to reconstruct? Is a held-out frog an anomaly to this network at all? Frog against truck: is the confusable class harder to detect, and by how much? The wide bottleneck: a better reconstruction of the normal data, but does it make a better detector, or does it reconstruct the anomalies better too? In one sentence: what does this

detector detect?

E25. A target the input does not contain (optional) ~1 min on a GPU

Run. `dn = train({"latent": 512, "epochs": 10, "noise": 0.2})`: noisy images in, clean images as the target. Compare with the plain $d = 512$ network of E22. Make noisy validation images the same way, `xn = x_va + 0.2 * torch.randn_like(x_va)`, and feed them to both.

Collect. one figure of 8 images: clean, noisy, the plain network on the noisy image, the denoiser on the noisy image; a table of the MSE to the *clean* image of the noisy input, the plain network's output and the denoiser's output; E23's classifier on the latent vectors of both networks.

Compute. one run of 10 epochs: ≈ 45 s on an Apple GPU, ≈ 6 min on an M5 CPU (or 5 epochs, 3 min).

Answer. Before running: does the plain network, which never saw a noisy image, remove any of the noise? By how much does the denoiser do better? Whose latent vector classifies better? Afterwards: what does the denoiser have to know about images to do its job that the plain network does not? Where in the plain network does the noise go?